

Dr. Wittawin Susutti

wittawin.sus@kmutt.ac.th

CSS234 WEB PROGRAMMING II

LECTURE 07 WEB WORKERS

Single-Threaded Nature of JS

JavaScript Execution Model

- Single-threaded language by design.
- Event Loop & Call Stack handle code execution.

The Main Thread Problem

- Responsible for executing JS
- Parsing HTML/CSS
- Rendering UI.
- 'Blocking the Main Thread' leads to unresponsive UI.

The Need for Background Processing

- Complex calculations and heavy data processing cause performance bottlenecks.

Introduction to Web Workers

What is a Web Worker?

- A JavaScript script that runs in a background thread, separate from the main execution thread of a web application.

Key Benefits

- Keeps the UI responsive.
- Utilizes multi-core processors effectively.

The Worker Global Scope

- Runs in a different global context than the window object.
- Accessed via 'self' inside the worker script.

Types of Web Workers

Dedicated Workers

- Linked to a single script (one-to-one relationship).

Shared Workers

- Shared across multiple scripts, if they are on the exact same domain.

Service Workers

- Acts as a network proxy.
- Enables offline capabilities, background sync, and push notifications.

Dedicated Workers - The Basics

Spawning a Dedicated Worker

- Constructor

```
const worker = new Worker('worker.js');
```

Message Passing System

- Communication via `postMessage()` method.
- Event listener: `onmessage` handles incoming data.

Data Serialization

- Data is copied, not shared (Structured Clone Algorithm).

Dedicated Workers - Code Example

Main.js (Main Thread)

```
const myWorker = new Worker('worker.js');  
myWorker.postMessage([10000000000, 20]);  
myWorker.onmessage = (e) => console.log(e.data);
```

Worker.js (Worker Thread)

```
onmessage = (e) => {  
    const result = doHeavyMath(e.data[0], e.data[1]);  
    postMessage(result);  
};
```

Terminating and Error Handling

Terminating a Worker

- Main Thread: `worker.terminate()` ;
- Inside Worker: `self.close()` ;
- Kills the thread immediately.

Error Handling

- Listen to `onerror` event.
- Provides: `message`, `filename`, and `lineno`.

Worker Context Limitations

What Workers CAN Do

- Network requests (`fetch`, `XMLHttpRequest`).
- WebSockets and IndexedDB.
- `setTimeout`, `setInterval`.

What Workers CANNOT Do

- Direct DOM Manipulation (Cannot access `document` or HTML elements).
- Access the `window` object.
- Access local UI interactions (click, scroll events).

Shared Workers - Concept

Definition

- A single worker script accessed by multiple windows, tabs, or iframes.

Same-Origin Policy

- All accessing scripts must be from the exact same origin.

Spawning a Shared Worker

```
const sharedWorker = new SharedWorker('shared-worker.js');
```

Shared Workers - Communication

The Port Object

- Communication happens via `MessagePort` objects.
- Must access `.port` property on the worker object.

Starting the Port

- Explicitly start: `worker.port.start()`.

Message Handling

- Listen for `onconnect` event inside Shared Worker.
- Extract port from the connection event.

Real-World Use Cases

Image & Video Processing

- Applying filters on canvas data.

Data Parsing

- Parsing massive JSON or CSV files.

Complex Calculations

- Cryptography, Pathfinding algorithms.

State Synchronization

- Synchronizing user state across tabs (Shared Workers).

Introduction to Service Workers

A Paradigm Shift

- A programmable network proxy between app and network.

Core Capabilities

- Offline First - Caching assets to load apps without internet.
- Background Sync - Deferring actions until stable connectivity.
- Push Notifications - Engaging users even when the tab is closed.

Security Requirement

- Must be served over HTTPS (or `localhost` for development).

Service Worker Scope

Scope Definition

- Determines which paths the Service Worker controls.
- Default is the directory where the SW file is located.

Example:

- If SW is at `/app/sw.js`, it controls `/app/`, `/app/users/`, etc.
- It does not control `/` (root) or `/about/`.

Best Practice:

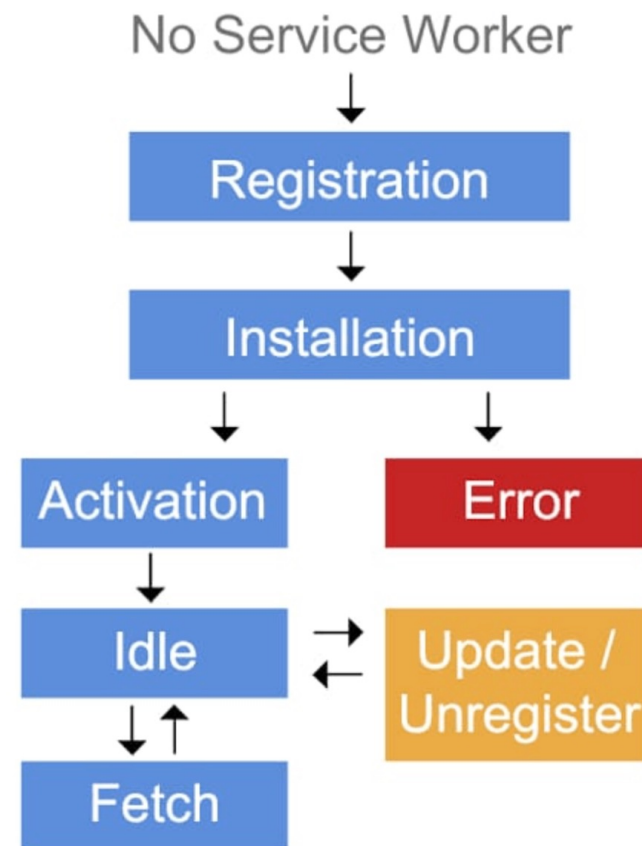
- Place the `sw.js` file at the root of your project directory.

The Service Worker Lifecycle

Why a Lifecycle?

- Enables 'offline-first' seamlessly.
- Allows updates without disrupting user experience.

Service worker states →



Phase 1: Registration

Registering the SW

```
if ('serviceWorker' in navigator) {  
  navigator.serviceWorker.register('/sw.js')  
    .then(reg => console.log('Registered!', reg.scope))  
    .catch(err => console.log('Failed!', err));  
}
```

Feature Detection

- Always check if the browser supports `serviceWorker`.
- Byte-Different Checking
 - The browser automatically checks if `sw.js` has changed (even by 1 byte) to trigger an update.

Phase 2: Installing

The First Event

- Fired when a SW is newly registered or updated.

Primary Use Case: Pre-caching

- Caching static HTML, CSS, JavaScript, and Images essential for the app shell.

`event.waitUntil()`

- Tells the browser to keep the installation phase alive until the promise (e.g., caching) resolves.

`self.skipWaiting()`

- Forces the waiting SW to become active immediately (use with caution).

Phase 3: Waiting

The Waiting Phase

- A new SW has installed successfully but an older SW is still controlling the page.

Why Wait?

- Ensures that only one version of your app is running at a time.
- Prevents mixing assets from V1 and V2, which could break the application.

Moving on

- The SW activates when all tabs using the old SW are completely closed or navigated away.

Phase 4: Activating

Taking Control

- Fired when the waiting SW finally takes over.

Primary Use Case: Cache Management

- Perfect time to delete old, outdated caches from previous versions.

`clients.claim()`

- Forces the SW to take control of uncontrolled clients (tabs) immediately after activation, without requiring a page refresh.

Intercepting Requests (fetch event)

The Heart of Offline Capabilities

- Fired every time a network request is made by the controlled page.

`event.respondWith()`

- Hijacks the browser's default network request.

Caching Strategies

- *Cache First, Network Fallback* (Good for static assets like images).
- *Network First, Cache Fallback* (Good for dynamic data like API responses).

Best Practices & Developer Tools

Development Tools

- Chrome DevTools -> Application Tab -> Service Workers.
- "Update on reload" checkbox (Essential for local development).

Best Practices

- Do not cache `sw.js` itself (browsers handle this, but server configurations should avoid max-age).
- Use libraries like Workbox to simplify caching strategies and SW management.